

S2D-99: Best Practice Summary

Series: S2D — Splunk to Dynatrace Migration | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/05/2026

Overview

This notebook consolidates every actionable best practice from the S2D (Splunk to Dynatrace) migration series into a single definitive reference. Each practice specifies the exact setting, value, or action to apply.

Table of Contents

- 1. [Migration Planning](#)
- 2. [Log Discovery and Validation](#)
- 3. [Query Translation \(SPL to DQL\)](#)
- 4. [Alert Migration - Anomaly Detectors](#)
- 5. [Alert Migration - Workflows](#)
- 6. [Extended Timeframes \(ArrayMovingSum\)](#)
- 7. [Metric Creation from Logs](#)
- 8. [Dashboard Migration](#)
- 9. [Naming Standards](#)
- 10. [DQL Performance and Syntax](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	<code>logs.read</code> , <code>settings.write</code> , <code>documents.write</code> , <code>automation.write</code>
Splunk Access	Ability to view existing queries, alerts, dashboards

Requirement	Details
Knowledge	Familiarity with both Splunk SPL and Dynatrace DQL

1. Migration Planning

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Follow the 5-phase migration sequence	Phase 1: Discovery, Phase 2: Data Validation, Phase 3: Query Translation, Phase 4: Alert Migration, Phase 5: Dashboard Migration	Critical	Process
2	Inventory all Splunk assets before starting	Document every dashboard, alert, report, and saved search with their SPL queries	Critical	Process
3	Prioritize by criticality	Migrate production monitoring first, then staging, then dev	Recommended	Process
4	Map Splunk indexes to Dynatrace buckets	Each Splunk index maps to a Grail bucket; document the mapping before translation	Critical	Process
5	Validate log availability before query translation	Run DQL validation queries (S2D-02) before attempting SPL-to-DQL conversion	Critical	Process
6	Compare log volumes between platforms	Run equivalent time-bounded queries in both Splunk and Dynatrace to confirm counts match	Recommended	Validation

2. Log Discovery and Validation

#	Best Practice	Recommended Setting/Value	Priority	Category
7	Set log ingest rules at the host-group level	Use host-group scope, not per-host	Recommended	Configuration
8	Enable OneAgent log content access	<code>oneagentctl --set-app-log-content-access=true</code> on every monitored host	Critical	Configuration
9	Configure custom log sources for non-standard paths	Settings > Log Monitoring > Custom log sources; specify the exact file path pattern	Recommended	Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
10	Configure timestamp and splitting rules for multi-line logs	Settings > Log Monitoring > Timestamp and splitting; set the timestamp pattern and line separator	Recommended	Configuration
11	Validate by host name first, then by log source path	Use <code>matchesPhrase(host.name, "...")</code> then <code>matchesPhrase(log.source, "...")</code>	Recommended	Validation
12	For Kubernetes, validate by cluster then namespace then deployment	Filter chain: <code>k8s.cluster.name -> k8s.namespace.name -> k8s.deployment.name</code>	Recommended	Validation
13	Remember: host-level OneAgent settings override environment settings	If log monitoring is disabled on the OneAgent, environment-level ingest rules will not apply	Critical	Configuration

3. Query Translation (SPL to DQL)

#	Best Practice	Recommended Setting/Value	Priority	Category
14	Use <code>==</code> for equality, not <code>=</code>	<code>filter field == "value"</code>	Critical	Syntax
15	Use double quotes only, not single quotes	<code>"value"</code> not <code>'value'</code>	Critical	Syntax
16	Use curly-brace arrays, not parentheses	<code>in(field, {"a", "b"})</code> not <code>field IN ('a', 'b')</code>	Critical	Syntax
17	Translate <code>search index=...</code> to fetch logs	<code>fetch logs, from:-1h</code> with explicit time range	Critical	Syntax
18	Translate <code>where</code> to <code>filter</code>	<code>filter field == "value"</code>	Critical	Syntax
19	Translate <code>stats</code> to <code>summarize</code>	<code>summarize count = count(), by: {field}</code>	Critical	Syntax
20	Translate <code>eval</code> to <code>fieldsAdd</code>	<code>fieldsAdd new_field = expression</code>	Critical	Syntax
21	Translate <code>table</code> to <code>fields</code>	<code>fields field1, field2</code>	Recommended	Syntax

#	Best Practice	Recommended Setting/Value	Priority	Category
22	Translate <code>sort -field</code> to <code>sort field desc</code>	Explicit <code>asc / desc</code> keyword	Recommended	Syntax
23	Translate <code>head N</code> to <code>limit N</code>	<code>limit 100</code>	Recommended	Syntax
24	Translate <code>rename old AS new</code> to <code>fieldsRename</code>	<code>fieldsRename old, alias:new</code>	Recommended	Syntax
25	Translate <code>timechart span=5m count by level</code> to <code>makeTimeseries</code>	<code>makeTimeseries count = count(), by:{loglevel}, interval:5m</code>	Critical	Syntax
26	Translate <code>rex field</code> extraction to <code>parse</code> with DPL	<code>parse content, "DATA? 'user=' WORD:username"</code> (lead with <code>DATA? :</code> <code>parse</code> anchors at the field start)	Recommended	Syntax
27	Translate <code>*value*</code> wildcard to <code>contains()</code>	<code>contains(field, "value", caseSensitive:false); matchesPhrase() matches whole words only</code>	Critical	Syntax
28	Translate <code>field IN ("a","b")</code> to <code>in()</code> function	<code>in(field, {"a", "b"})</code>	Critical	Syntax
29	Translate <code>NOT field="value"</code> to <code>!=</code> or <code>filterOut</code>	<code>filter field != "value" or filterOut field == "value"</code>	Recommended	Syntax
30	Always alias aggregation results	<code>summarize count = count() not summarize count()</code>	Critical	Syntax

4. Alert Migration - Anomaly Detectors

#	Best Practice	Recommended Setting/Value	Priority	Category
31	Apply the threshold translation formula	<code>Splunk Threshold = DT Threshold x DT Violating Samples</code>	Critical	Alerting

#	Best Practice	Recommended Setting/Value	Priority	Category
32	Use the "Alert Reimagined" strategy as default	Threshold: calculated, Sliding Window: match Splunk timeframe, Violating Samples: 2-3, Dealerting Samples: how many back-to-normal samples close the event	Recommended	Alerting
33	Set Anomaly Detector query interval to 1 minute	<code>interval:1m</code> in <code>makeTimeseries</code>	Critical	Alerting
34	Set the sliding window to match Splunk query timeframe	Max: 60 minutes. If Splunk timeframe is 15m, set sliding window to 15	Critical	Alerting
35	Do not map Splunk suppression onto dealerting samples	Dealerting controls when an open event closes; Splunk suppress controls re-notification. Handle re-notification in the routing workflow	Recommended	Alerting
36	Use <code>by: {dt.entity.cloud_application}</code> for entity association	Ensures alerts are tied to the correct monitored entity	Critical	Alerting
37	Prefer Anomaly Detectors over Workflows when timeframe is 60 min or less	Per-minute evaluation and Davis correlation; queries are billed, so use Delay to reduce executions	Critical	Alerting
38	If alert timeframe exceeds 60 minutes, use Workflows or <code>ArrayMovingSum</code>	Anomaly Detectors max sliding window = 60 minutes	Critical	Alerting
39	Use the data-driven strategy when historical data is available	Analyze per-minute samples (24 h or less per query at 1-minute resolution; repeat for several days)	Recommended	Alerting
40	Validate the alert query returns timeseries data before configuring	Run the query in a notebook first; confirm <code>makeTimeseries</code> output	Critical	Validation

5. Alert Migration - Workflows

#	Best Practice	Recommended Setting/Value	Priority	Category
41	Use Workflows only when: timeframe > 60 min, alert runs few times/day, business-hours only, or is actually a report	Otherwise use Anomaly Detectors	Critical	Alerting
42	Specify timeframe explicitly in the DQL query, not from dashboard	<code>fetch logs, from:now()-7d</code> in the workflow query itself	Critical	Alerting
43	Workflow queries do NOT require <code>makeTimeseries</code> output	Use <code>summarize</code> to return a single aggregated value	Recommended	Alerting
44	Set event timeout in the JavaScript event creation step	<code>timeout: 15</code> (minutes) to auto-expire events	Recommended	Alerting
45	Workflow events do NOT auto-close or auto-update	Design event lifecycle accordingly; events expire by timeout only	Critical	Alerting
46	Workflow events are NOT correlated by Dynatrace Intelligence	No root-cause analysis or problem grouping	Recommended	Alerting
47	Account for workflow license consumption	Every standard workflow is billed per hour it exists, regardless of how often it runs; the DQL it executes is billed separately as query consumption	Recommended	Licensing
48	For business-hours-only alerting, filter by hour in the Dynatrace Intelligence query instead of using a Workflow	<code>fieldsAdd hour = getHour(timestamp, timezone:"America/New_York") then filter hour >= 8 and hour < 18</code> (queries run in UTC; set your time zone)	Recommended	Alerting

6. Extended Timeframes (ArrayMovingSum)

#	Best Practice	Recommended Setting/Value	Priority	Category
49	Use <code>arrayMovingSum(array, 60)</code> with <code>interval:1m</code> for 60-minute rolling sum	Each data point = sum of preceding 60 minutes	Critical	DQL
50	Max <code>arrayMovingSum</code> window = 60	Cannot exceed 60 data points regardless of interval	Critical	DQL
51	With Anomaly Detectors + <code>ArrayMovingSum</code> , set sliding window = 1 and violating samples = 1	Each data point already contains the full rolling aggregation	Critical	Alerting
52	Set threshold to total count expected in the rolling window	If Splunk threshold = 500 errors in 60 min, set DT threshold = 500	Critical	Alerting
53	Remove the original timeseries field after adding the rolling sum	<code>fieldsRemove error_count</code> after <code>fieldsAdd error_count_1h = arrayMovingSum(error_count, 60)</code>	Recommended	DQL
54	For dashboard visualizations > 60 min, increase the interval	<code>interval:5m</code> with <code>window:48</code> = 4-hour rolling sum (intervals snap to supported values; check the returned <code>interval</code>)	Recommended	DQL
55	For alerts > 60 minutes that cannot use proportional reduction, use Workflows	If error distribution is uneven, proportional threshold reduction will produce false positives	Recommended	Alerting

7. Metric Creation from Logs

#	Best Practice	Recommended Setting/Value	Priority	Category
56	Create log-based metrics for frequently executed dashboard and alert queries	Reduces query cost, improves speed, increases retention	Recommended	Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Metric filter must use only supported functions	<code>matchesPhrase</code> , <code>matchesValue</code> , <code>isNull</code> , <code>isNotNull</code> , basic operators only. No <code>parse</code> or <code>fieldsAdd</code>	Critical	Configuration
58	Avoid high-cardinality dimensions	Never use transaction IDs, session IDs, request IDs, or timestamps as metric dimensions	Critical	Configuration
59	Consolidate similar metrics using dimensions instead of separate metrics	One <code>service_errors</code> metric with <code>k8s.deployment.name</code> dimension, not per-service metrics	Recommended	Design
60	Name metrics as <code>log.[app_name].[metric_description]</code>	Example: <code>log.easytravel.error_count</code>	Recommended	Naming
61	Complete all SPL-to-DQL translation before requesting metric extraction	Metric filters must match the final DQL query filters	Critical	Process
62	Query extracted metrics with <code>timeseries</code> not <code>fetch logs</code>	<code>timeseries</code> <code>sum(log.easytravel.error_count),</code> <code>from:-1h</code> (counter: use <code>sum()</code> , not <code>avg()</code>)	Critical	Syntax

8. Dashboard Migration

#	Best Practice	Recommended Setting/Value	Priority	Category
63	Layout: top row = single-value KPIs, middle = time-series trends, bottom = detail tables	Standard 3-tier dashboard layout	Recommended	Design
64	Use dashboard variables for interactive filtering	Define <code>\$cluster</code> , <code>\$namespace</code> , <code>\$deployment</code> variables with query-backed dropdowns	Recommended	Design
65	Populate dropdown variables with DQL	<code>fetch logs, from:-1h \</code> <code>summarize count(), by:{field}</code> <code>\ fields field \ sort field</code> <code>asc</code>	Recommended	Design

#	Best Practice	Recommended Setting/Value	Priority	Category
66	Use consistent time intervals across related charts on the same dashboard	All trend charts should use the same interval: value	Recommended	Design
67	Map Splunk gauges to the Gauge or Meter bar visualization	Radial gauge → Gauge; filler/marker gauge → Meter bar or Gauge	Recommended	Design
68	Create Log Searcher dashboards for investigation	VM version: variables for host_name and log_source . K8s version: variables for cluster , namespace , deployment	Recommended	Design
69	Validate migrated dashboard data against Splunk	Compare identical time ranges side-by-side before decommissioning Splunk dashboards	Critical	Validation

9. Naming Standards

#	Best Practice	Recommended Setting/Value	Priority	Category
70	Dashboard names	[app_name] Dashboard Title	Critical	Naming
71	Alert configuration names	[app_name] alert name from splunk	Critical	Naming
72	Alert event names (triggered title)	[app_name] [priority] - {dimension} - alert name e.g. [EasyTravel] P2 - {dims:host.name} - High Error Count	Critical	Naming
73	Report names	[Report] [app_name] Report Title	Recommended	Naming
74	Lookup table paths	/lookups/[app_name]/[table_name]	Recommended	Naming
75	Log-based metric names	log.[app_name].[metric_description] (lowercase, underscores)	Recommended	Naming
76	Alerting workflow names	[Alert] [app_name] Alert Description	Recommended	Naming
77	Automation workflow names	[Auto] [app_name] Automation Description	Recommended	Naming

#	Best Practice	Recommended Setting/Value	Priority	Category
78	Remediation workflow names	<code>[Remediate] [app_name] Remediation Description</code>	Optional	Naming
79	Enforce naming during migration, not after	Review every asset name at creation time	Critical	Process
80	Use infrastructure-as-code to enforce naming	Monaco or Terraform templates with naming prefixes built in	Recommended	Process

10. DQL Performance and Syntax

#	Best Practice	Recommended Setting/Value	Priority	Category
81	Always specify a time range on <code>fetch</code>	<code>fetch logs, from:-1h</code> (never bare <code>fetch logs</code>)	Critical	Performance
82	Filter immediately after <code>fetch</code>	Place <code>filter</code> commands before any <code>fieldsAdd</code> , <code>parse</code> , or <code>summarize</code>	Critical	Performance
83	Use <code>==</code> for exact matches, <code>matchesPhrase</code> for token search	<code>==</code> is faster than <code>~</code> or <code>matchesPhrase</code> when the full value is known	Recommended	Performance
84	Target specific Grail buckets	<code>fetch logs, bucket: {"app_logs_*"}</code> to avoid scanning all buckets	Recommended	Performance
85	Use named parameters for DQL functions	<code>round(value, decimals: 2)</code> not <code>round(value, 2)</code>	Critical	Syntax
86	Use <code>countIf()</code> instead of nested filter + count	<code>summarize errors = countIf(status == "ERROR")</code> in a single pass	Recommended	Performance
87	Check <code>isNotNull()</code> before aggregating optional fields	<code>filter isNotNull(db.system)</code> before <code>summarize avg(duration), by: {db.system}</code>	Recommended	Correctness
88	Sort and limit must come last in the pipeline	<code>sort field desc \ limit 10</code> as final commands	Critical	Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
89	Use <code>fieldsKeep</code> or <code>fieldsRemove</code> early to drop unneeded columns	Reduces data volume through the pipeline	Recommended	Performance
90	Use <code>samplingRatio</code> for exploratory queries on large datasets	<code>fetch logs, samplingRatio:100</code> then multiply results back	Optional	Performance

Summary

This notebook contains **90 best practices** across 10 categories for Splunk to Dynatrace migration:

Category	Count	Priority Breakdown
Migration Planning	6	4 Critical, 2 Recommended
Log Discovery and Validation	7	3 Critical, 4 Recommended
Query Translation (SPL to DQL)	17	10 Critical, 7 Recommended
Alert Migration - Anomaly Detectors	10	7 Critical, 3 Recommended
Alert Migration - Workflows	8	3 Critical, 5 Recommended
Extended Timeframes (<code>ArrayMovingSum</code>)	7	4 Critical, 3 Recommended
Metric Creation from Logs	7	3 Critical, 4 Recommended
Dashboard Migration	7	1 Critical, 6 Recommended
Naming Standards	11	4 Critical, 6 Recommended, 1 Optional
DQL Performance and Syntax	10	4 Critical, 5 Recommended, 1 Optional
Total	90	43 Critical, 45 Recommended, 2 Optional

References

- **S2D-01** - Getting Started: Migration overview and planning
- **S2D-02** - Locating Logs: Data validation and ingest configuration
- **S2D-03** - SPL to DQL: Query translation fundamentals
- **S2D-04** - Anomaly Detectors: Continuous alert migration
- **S2D-05** - Workflow Alerts: Scheduled and extended alerting
- **S2D-06** - `ArrayMovingSum`: Extended timeframe handling
- **S2D-07** - Metric Creation: `OpenPipeline` log-based metrics

- **S2D-08** - Dashboard Migration: Visualization conversion
- **S2D-09** - Naming Standards: Asset organization conventions

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.